DevOps Practical Exam - Questions & Solutions

Topic-wise: Shell Scripting, Git/GitHub, GitHub Actions, Docker, Compose, Jenkins, Viva

DevOps Study Notes

2026

Contents

How to Use This Document **3**

1. Shell Scripting **4**
- S1. Factorial of a number . . . 4
- S2. Perfect number . . . 4
- S3. Count lines and words in a file . . . 4
- S4. Count words, lines, characters (file name from command line) . . . 5
- S5. Pattern 0 / 0 1 / 0 1 2 / 0 1 2 3 . . . 5
- S6. Pattern 1 / 1 2 1 / 1 3 3 1 (Pascal's triangle) . . . 5
- S7. Delete all empty lines from a file . . . 6
- S8. Reverse a string . . . 6
- S9. Maximum of three numbers . . . 6
- S10. Reverse array elements . . . 7
- S11. Delete all text files from a directory . . . 7
- S12. Compile all C files and delete those with errors . . . 7
- S13. Pipe command and changing the system date . . . 8

2. Git & GitHub **9**
- G1. Clone a public repository and pull files . . . 9
- G2. Create a GitHub tag . . . 9
- G3. Run any 4 GitHub commands and print logs . . . 9
- G4. Create a webhook for GitHub using ngrok . . . 9

3. GitHub Actions **11**
- A1. Run Node.js tests on every push to main . . . 11
- A2. Python project - clean package . . . 11
- A3. Python project - run JUnit tests and generate a report . . . 12
- A4. Node.js lint errors stored in a file . . . 13

4. Docker **15**
- D1. Dockerfile to build a Node.js container . . . 15
- D2. Build and run the container; open it in the browser . . . 15
- D3. Pull any image from Docker Hub and run it . . . 16
- D4. docker exec, docker rm, docker rmi . . . 16

5. Docker Compose **17**
- C1. Docker Compose file with a MySQL server . . . 17
- C2. Docker Compose file for MySQL user creation . . . 17

6. Jenkins **19**
J1. Check out code from GitHub and start a Node app 19
J2. Node project - install dependencies 19
J3. Node project - start the application 20
J4. Python project - send reports by email 21

7. Viva Questions and Answers (Q3 - 10 Marks) **22**
Shell Scripting 22
Git & GitHub 22
GitHub Actions 22
Docker 22
Docker Compose 23
Jenkins 23

How to Use This Document

The question papers repeat the same questions across different sets, so they are **grouped by topic** here. Each distinct question is listed once with its marks, followed by its solution. If your paper has the same question, use the matching solution below.

Every paper has the same pattern: **Q1 (15 marks)**, **Q2 (25 marks)**, **Q3 Viva (10 marks)**. The viva questions for all topics are in Section 7.

Topic	Questions covered
1. Shell Scripting	S1-S13 (factorial, perfect number, word count, patterns, file/array/string handling, compile C, pipe & date)
2. Git & GitHub	G1-G4 (clone & pull, tag, 4 commands with logs, ngrok webhook)
3. GitHub Actions	A1-A4 (Node tests, Python clean package, Python JUnit, Node lint)
4. Docker	D1-D4 (Dockerfile, build & run, pull & run, exec/rm/rmi)
5. Docker Compose	C1-C2 (MySQL server, MySQL user creation)
6. Jenkins	J1-J4 (checkout & start app, install deps, start app, Python email report)
7. Viva	Q&A for every topic

What was tested. All 13 shell scripts, the Git commands, the ESLint / pytest / Python-build commands used by the workflows, the YAML files, the Docker Compose files (syntax) and the Node server were run on a real machine; the outputs shown are genuine. These could **not** be run here and follow the standard documented behaviour: Docker commands (no Docker daemon available), Jenkins pipelines, the workflows on GitHub itself, `ngrok`, and `sudo date -s` (it would change the machine's clock). Runnable copies of all files are in the `exam-solutions/` folder.

1. Shell Scripting

Run any script: save it as `name.sh`, then `chmod +x name.sh` and `./name.sh`.

S1. Factorial of a number

Question: Write a shell script to write a factorial of a number. *(15 Marks)*

```
#!/bin/bash
read -p "Enter a number: " n
fact=1
for ((i=2; i<=n; i++)); do
    fact=$((fact * i))
done
echo "Factorial of $n is $fact"
```

Output (input 5): `Factorial of 5 is 120`

S2. Perfect number

Question: Write a shell script to check whether the number is perfect or not. *(15 Marks)*

A perfect number equals the sum of its proper divisors (28 = 1+2+4+7+14).

```
#!/bin/bash
read -p "Enter a number: " n
sum=0
for ((i=1; i<n; i++)); do
    if (( n % i == 0 )); then
        sum=$((sum + i))
    fi
done
if (( sum == n && n > 0 )); then
    echo "$n is a Perfect number"
else
    echo "$n is NOT a Perfect number"
fi
```

Output: input 28 -> `28 is a Perfect number`; input 12 -> `12 is NOT a Perfect number`

S3. Count lines and words in a file

Question: Create a shell script to count the number of lines and words in a file. *(15 Marks)*

```
#!/bin/bash
read -p "Enter file name: " file
if [ ! -f "$file" ]; then
    echo "File not found: $file"
    exit 1
fi
echo "Lines : $(wc -l < "$file")"
echo "Words : $(wc -w < "$file")"
```

Output (for a 4-line, 9-word file): `Lines : 4` / `Words : 9`

S4. Count words, lines, characters (file name from command line)

Question: Write a shell script to count the number of words, lines and characters from a given file; accept the file name from the command line. *(15 Marks)*

```
#!/bin/bash
if [ $# -ne 1 ]; then
    echo "Usage: $0 <filename>"
    exit 1
fi
file="$1"
if [ ! -f "$file" ]; then
    echo "File not found: $file"
    exit 1
fi
echo "Lines      : $(wc -l < "$file")"
echo "Words      : $(wc -w < "$file")"
echo "Characters : $(wc -m < "$file")"
```

Run: `./wc_args.sh sample.txt` -> `Lines : 4`, `Words : 9`, `Characters : 46`. Running it without a file name prints the usage message.

S5. Pattern 0 / 0 1 / 0 1 2 / 0 1 2 3

Question: Write a shell script to print the pattern `0`, `0 1`, `0 1 2`, `0 1 2 3`. *(25 Marks)*

```
#!/bin/bash
for ((i=0; i<4; i++)); do
    for ((j=0; j<=i; j++)); do
        echo -n "$j "
    done
    echo
done
```

Output:

```
0
0 1
0 1 2
0 1 2 3
```

S6. Pattern 1 / 1 2 1 / 1 3 3 1 (Pascal's triangle)

Question: Write a shell script to print the pattern 1, `1 2 1`, `1 3 3 1`. *(25 Marks)*

This is Pascal's triangle (the paper's pattern skips the `1 1` row; the script below prints the full triangle, which includes the rows asked for).

```
#!/bin/bash
rows=4
for ((i=0; i<rows; i++)); do
    c=1
    for ((j=0; j<=i; j++)); do
        echo -n "$c "
        c=$(( c * (i - j) / (j + 1) ))
    done
```

```
    echo
done
```

Output:

```
1
1 1
1 2 1
1 3 3 1
```

S7. Delete all empty lines from a file

Question: Write a shell script to delete all empty lines from any file. *(15 Marks)*

```
#!/bin/bash
read -p "Enter file name: " file
if [ ! -f "$file" ]; then
    echo "File not found: $file"
    exit 1
fi
sed -i '/^[[:space:]]*$/d' "$file"
echo "Empty lines removed from $file"
cat "$file"
```

Output: a file with a blank line between text lines is printed back with no blank lines. (`sed -i '/^$/d'` is the short form of the same command.)

S8. Reverse a string

Question: Write a shell script to reverse a string. *(15 Marks)*

```
#!/bin/bash
read -p "Enter a string: " str
rev_str=""
for ((i=${#str}-1; i>=0; i--)); do
    rev_str="$rev_str${str:$i:1}"
done
echo "Reversed: $rev_str"
```

Output (input `DevOps`): `Reversed: spOveD`

S9. Maximum of three numbers

Question: Write a shell script to find the max of 3 numbers. *(15 Marks)*

```
#!/bin/bash
read -p "Enter three numbers: " a b c
max=$a
(( b > max )) && max=$b
(( c > max )) && max=$c
echo "Maximum is $max"
```

Output (input `12 45 7`): `Maximum is 45`

S10. Reverse array elements

Question: Write a shell script to reverse array elements. *(15 Marks)*

```
#!/bin/bash
arr=(10 20 30 40 50)
echo "Original: ${arr[@]}"
rev=()
for ((i=${#arr[@]}-1; i>=0; i--)); do
    rev+=("${arr[$i]}")
done
echo "Reversed: ${rev[@]}"
```

Output:

```
Original: 10 20 30 40 50
Reversed: 50 40 30 20 10
```

S11. Delete all text files from a directory

Question: Write a shell script to delete all text files from the directory. *(15 Marks)*

```
#!/bin/bash
read -p "Enter directory path: " dir
if [ ! -d "$dir" ]; then
    echo "Directory not found: $dir"
    exit 1
fi
echo "Deleting these files:"
ls "$dir"/*.txt 2>/dev/null || { echo "No .txt files found"; exit 0; }
rm -f "$dir"/*.txt
echo "Done."
```

Output: lists `a.txt`, `b.txt`, deletes them; `keep.log` is untouched.

S12. Compile all C files and delete those with errors

Question: Write a shell script to compile all C files and delete those files having errors. *(25 Marks)*

```
#!/bin/bash
for f in *.c; do
    [ -e "$f" ] || { echo "No .c files found"; exit 0; }
    if gcc "$f" -o "${f%.c}.out" 2>/dev/null; then
        echo "OK      : $f compiled"
    else
        echo "ERROR   : $f has errors -> deleting"
        rm -f "$f"
    fi
done
```

Output (one valid file, one with a missing semicolon):

```
ERROR   : bad.c has errors -> deleting
OK      : good.c compiled
```

`bad.c` is removed; `good.c` and its executable `good.out` remain.

S13. Pipe command and changing the system date

Question: Write a shell script to illustrate the use of the pipe command and change the system date. *(15 Marks)*

A **pipe (|)** sends the output of one command as the input of the next.

```
#!/bin/bash
echo "--- Pipe demo ---"
ls -l | wc -l                                  # count entries
ps aux | grep bash | head -3                   # first 3 bash processes
cat /etc/passwd | cut -d: -f1 | sort | head -5   # first 5 usernames, sorted

echo "--- Date demo ---"
echo "Current date: $(date)"
sudo date -s "2026-12-25 10:30:00"             # needs root
echo "New date    : $(date)"
```

Output of the pipe part: 6 (entry count) and the first usernames sorted (`_apt`, `asif`, `avahi`, backup, `bin`). **Note:** `sudo date -s` changes the real system clock, so it was not run during testing; run it only on a practice machine and reset with `sudo timedatectl set-ntp true`.

2. Git & GitHub

G1. Clone a public repository and pull files

Question: Clone an existing public repository and pull files from it. *(15 Marks)*

```
git clone https://github.com/octocat/Hello-World.git
cd Hello-World
git pull
git log --oneline | head -3
```

Output:

```
Cloning into 'Hello-World'...
Already up to date.
7fd1a60 Merge pull request #6 from Spaceghost/patch-1
7629413 New line at end of file. --Signed off by Spaceghost
553c207 first commit
```

`git clone` copies the whole repository; `git pull` later fetches and merges any new commits from the remote.

G2. Create a GitHub tag

Question: Create a GitHub tag for any repository. *(15 Marks)*

```
git tag v1.0                          # lightweight tag
git tag -a v1.1 -m "Release 1.1"    # annotated tag (has author, date, message)
git tag                               # list tags
git push origin v1.0 v1.1             # publish tags to GitHub
```

Output of `git tag`: `v1.0` and `v1.1`. After the push, the tags appear under the repository's *Releases / Tags* page on GitHub.

G3. Run any 4 GitHub commands and print logs

Question: Run any 4 GitHub commands and print the logs from them. *(15 Marks)*

```
git status -sb          # 1. working tree status
git branch              # 2. list branches
git log --oneline       # 3. commit history (the "log")
git diff HEAD~1 --stat  # 4. what changed in the last commit
```

Output:

```
## main
* main
f102ea9 second commit
f474ce1 first commit
 a.txt | 1 +
 1 file changed, 1 insertion(+)
```

To save the log to a file as the question asks: `git log > git_log.txt`.

G4. Create a webhook for GitHub using ngrok

Question: Create a webhook for GitHub using ngrok. *(15 Marks)*

A webhook makes GitHub send an HTTP POST to a URL whenever an event (such as a push) happens. GitHub cannot reach `localhost`, so **ngrok** creates a public HTTPS URL that tunnels to a port on your machine.

Step 1 - a small receiver (`webhook_server.js`, tested):

```
const http = require('http');
http.createServer((req, res) => {
  if (req.method === 'POST' && req.url === '/webhook') {
    let body = '';
    req.on('data', chunk => (body += chunk));
    req.on('end', () => {
      const event = req.headers['x-github-event'] || 'unknown';
      let repo = 'unknown';
      try { repo = JSON.parse(body).repository.full_name; } catch (e) {}
      console.log(`Received "${event}" event from ${repo}`);
      res.writeHead(200);
      res.end('OK');
    });
  } else {
    res.writeHead(200);
    res.end('Webhook server is running');
  }
}).listen(3000, () => console.log('Listening on port 3000'));
```

Step 2 - run it and start the tunnel:

```
node webhook_server.js
ngrok config add-authtoken <your-token>     # once, from ngrok.com dashboard
ngrok http 3000
```

ngrok prints a forwarding URL such as `https://abcd-1234.ngrok-free.app`.

Step 3 - add the webhook on GitHub: repository -> *Settings* -> *Webhooks* -> *Add webhook*:

- Payload URL: `https://abcd-1234.ngrok-free.app/webhook`
- Content type: `application/json`
- Events: *Just the push event*

Step 4 - verify: push a commit. The terminal prints `Received "push" event from <user>/<repo>`; GitHub's *Recent Deliveries* tab shows a green tick (HTTP 200), and the ngrok inspector at `http://127.0.0.1:4040` shows the request.

Tested locally: a simulated POST printed `Received "push" event from asif/demo`. ngrok itself needs an account/token, so the tunnel step was not run here.

3. GitHub Actions

Workflow files live in `.github/workflows/` inside the repository. A workflow runs on GitHub's servers automatically when its trigger happens.

A1. Run Node.js tests on every push to main

Question: Create GitHub Actions that run every time the code is pushed to the main branch and run Node.js tests, e.g. `npm run test`. *(25 Marks)*

`.github/workflows/node-test.yml`:

```yaml
name: Node.js Tests

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: '20'

      - name: Install dependencies
        run: npm ci

      - name: Run tests
        run: npm run test
```

The project's `package.json` must contain `"test": "..."` under `scripts`. **Result:** after `git push origin main`, open the repository's **Actions** tab; the run shows a green tick when tests pass.

A2. Python project - clean package

Question: Consider the given Python project available on your machine and create a GitHub pipeline to demonstrate clean package. *(25 Marks)*

"Clean package" = remove old build output, then build a fresh package.

```yaml
name: Python Clean Package

on:
  push:
    branches: [main]

jobs:
  clean-package:
```

```yaml
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install build tool
        run: pip install build

      - name: Clean old build output
        run: rm -rf build/ dist/ *.egg-info

      - name: Build package
        run: python -m build

      - name: Show built package
        run: ls -l dist/
```

Tested output of the build commands: `Successfully built mypkg-0.1.0.tar.gz` and `mypkg-0.1.0-py3-none-any.whl`, and `dist/` contains both files. The project needs a `pyproject.toml` (or `setup.py`).

A3. Python project - run JUnit tests and generate a report

Question: Consider the given Python project and create a GitHub pipeline to demonstrate running JUnit tests and generating a report. *(25 Marks)*

pytest can write results in JUnit XML format.

```yaml
name: Python JUnit Report

on:
  push:
    branches: [main]

jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4

      - uses: actions/setup-python@v5
        with:
          python-version: '3.12'

      - name: Install dependencies
        run: pip install -r requirements.txt pytest

      - name: Run tests and generate JUnit XML report
        run: pytest --junitxml=report.xml

      - name: Upload test report
```

```
        if: always()
        uses: actions/upload-artifact@v4
        with:
          name: junit-report
          path: report.xml
```

Tested output: `1 passed` and `report.xml` containing `<testsuite ... tests="1" failures="0" errors="0">`. After the run, download `junit-report` from the workflow run's *Artifacts* section.

A4. Node.js lint errors stored in a file

Question: Create a GitHub workflow that runs Node.js lint errors and stores them in a file. *(25 Marks)*

ESLint v9 needs a config file, `eslint.config.js`:

```
module.exports = [
  {
    files: ['**/*.js'],
    languageOptions: {
      sourceType: 'commonjs',
      globals: { require: 'readonly', module: 'readonly',
                 console: 'readonly', process: 'readonly' }
    },
    rules: { 'no-unused-vars': 'error', 'semi': ['error', 'always'] },
  },
];
```

Workflow (`npm i -D eslint` must be in the project):

```
name: Node.js Lint

on:
  push:
    branches: [main]

jobs:
  lint:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with:
          node-version: '20'
      - run: npm ci

      - name: Run ESLint and save errors to a file
        run: npx eslint . -o lint-report.txt || true

      - name: Show the report
        run: cat lint-report.txt

      - name: Upload lint report
```

```
        uses: actions/upload-artifact@v4
        with:
          name: lint-report
          path: lint-report.txt
```

`-o lint-report.txt` writes the errors to the file; `|| true` stops the step from failing so the report can still be uploaded.

Tested output (`lint-report.txt`):

```
app.js
  1:7   error  'unused' is assigned a value but never used  no-unused-vars
  1:17  error  Missing semicolon                            semi
  2:18  error  Missing semicolon                            semi

3 problems (3 errors, 0 warnings)
```

4. Docker

Setup used by the Docker questions - a tiny Node app (`server.js`, tested locally and it answers `Hello from Node.js running in Docker!`):

```
const http = require('http');
const PORT = process.env.PORT || 3000;

http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/html' });
  res.end('<h1>Hello from Node.js running in Docker!</h1>');
}).listen(PORT, '0.0.0.0', () => console.log(`Server running on ${PORT}`));
```

`package.json`:

```
{
  "name": "docker-node-demo",
  "version": "1.0.0",
  "main": "server.js",
  "scripts": { "start": "node server.js" }
}
```

D1. Dockerfile to build a Node.js container

Question: Create a Dockerfile to build a Node.js container. *(25 Marks)*

`Dockerfile`:

```
FROM node:20-alpine
WORKDIR /app
COPY package*.json ./
RUN npm install
COPY . .
EXPOSE 3000
CMD ["node", "server.js"]
```

Line	Meaning
`FROM`	Base image (Node 20 on small Alpine Linux)
`WORKDIR`	Folder inside the image where commands run
`COPY package*.json` + `RUN npm install`	Install dependencies first so Docker can cache this layer
`COPY . .`	Copy the application code
`EXPOSE 3000`	Documents the port the app listens on
`CMD`	Command that starts the app when the container runs

```
docker build -t node-app .
```

D2. Build and run the container; open it in the browser

Question: Build and run a Docker container to ensure the Node.js app is running and accessible from the browser. *(25 Marks)*

```
docker build -t node-app .                  # build the image
docker run -d --name node-app -p 3000:3000 node-app    # run it
docker ps                                   # confirm it is "Up"
curl http://localhost:3000                  # or open in the browser
docker logs node-app                       # shows: Server running on port 3000
```

`-p 3000:3000` maps host port 3000 to container port 3000; `-d` runs it in the background. **Expected result:** the browser at `http://localhost:3000` shows **"Hello from Node.js running in Docker!"**.

D3. Pull any image from Docker Hub and run it

Question: Pull any Docker image from the hub and run it. *(25 Marks)*

```
docker pull nginx:alpine                        # download from Docker Hub
docker images                                   # confirm it is stored locally
docker run -d --name web -p 8080:80 nginx:alpine
docker ps                                       # shows web, port 8080->80
curl http://localhost:8080                      # nginx welcome page
```

Expected result: `http://localhost:8080` shows *"Welcome to nginx!"*. (`docker run hello-world` is a smaller alternative that just prints a message.)

D4. docker exec, docker rm, docker rmi

Question: Execute the Docker commands `docker exec`, `docker rm`, `docker rmi`. *(25 Marks)*

Using the nginx container from D3:

```
docker exec -it web sh          # open a shell INSIDE the running container
#   / # ls /usr/share/nginx/html   (run commands, then type: exit)
docker exec web nginx -v        # run a single command inside it

docker stop web                 # a running container must be stopped first
docker rm web                   # remove the container
docker rm -f web               # (alternative) force-remove a running container

docker rmi nginx:alpine         # remove the IMAGE (only if no container uses
 ↪  it)
docker images                   # nginx:alpine is gone
```

Command	What it does
`docker exec`	Runs a command inside a **running container**
`docker rm`	Deletes a **container**
`docker rmi`	Deletes an **image**

5. Docker Compose

Docker Compose describes containers in one YAML file and starts them with `docker compose up -d`. Both files below were syntax-checked with `docker compose config`.

C1. Docker Compose file with a MySQL server

Question: Create a Docker Compose file to contain a MySQL server. *(25 Marks)*

`docker-compose.yml`:

```yaml
services:
  mysql:
    image: mysql:8.0
    container_name: mysql-server
    restart: unless-stopped
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: appdb
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql

volumes:
  mysql_data:
```

```
docker compose up -d
docker compose ps
docker exec -it mysql-server mysql -uroot -prootpass -e "SHOW DATABASES;"
docker compose down            # stop (data stays in the mysql_data volume)
```

The `volumes` entry keeps the database files even if the container is removed.

C2. Docker Compose file for MySQL user creation

Question: Write a Docker Compose file for MySQL user creation; pull any Docker image from the hub and run it. *(25 Marks)*

The official MySQL image creates a user automatically from the `MYSQL_USER` / `MYSQL_PASSWORD` variables on first start.

```yaml
services:
  mysql:
    image: mysql:8.0
    container_name: mysql-server
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: appdb
      MYSQL_USER: appuser
      MYSQL_PASSWORD: apppass
    ports:
      - "3306:3306"
    volumes:
      - mysql_data:/var/lib/mysql
```

```
volumes:
  mysql_data:
```

```
docker compose pull            # pulls mysql:8.0 from Docker Hub
docker compose up -d           # runs it
docker exec -it mysql-server mysql -uappuser -papppass -e "SHOW DATABASES;"
```

Expected result: logging in as appuser works and shows the appdb database. To create more users manually: `docker exec -it mysql-server mysql -uroot -prootpass`, then `CREATE USER 'bob'@'%' IDENTIFIED BY 'bobpass'; GRANT ALL ON appdb.* TO 'bob'@'%';`. The env variables only take effect the first time (when the data volume is empty); run `docker compose down -v` to reset.

6. Jenkins

One-time setup: install and start Jenkins (for example `docker run -d -p 8080:8080 -v jenkins_home:/var/jenkins_home jenkins/jenkins:lts`), open `http://localhost:8080`, install the suggested plugins. For each question: **New Item -> Pipeline**, paste the script under *Pipeline -> Pipeline script*, **Save**, **Build Now**. Replace the `<your-...>` placeholders with your repository URL. Node.js (and Python for J4) must be installed on the Jenkins machine.

J1. Check out code from GitHub and start a Node app

Question: Create a Jenkins pipeline to check out code from GitHub and start a Node app. *(15 Marks)*

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url:
 ↪  'https://github.com/<your-username>/<your-node-repo>.git'
            }
        }
        stage('Install') {
            steps { sh 'npm install' }
        }
        stage('Start App') {
            steps {
                withEnv(['BUILD_ID=dontKillMe',
                 ↪  'JENKINS_NODE_COOKIE=dontKillMe']) {
                    sh 'nohup node server.js > app.log 2>&1 &'
                }
                sh 'sleep 3 && curl -s http://localhost:3000'
            }
        }
    }
}
```

`BUILD_ID=dontKillMe` stops Jenkins from killing the background app when the build finishes. **Expected console output:** the repository is cloned, `npm install` runs, and `curl` prints the app's response.

J2. Node project - install dependencies

Question: Consider the given Node.js project available on your machine and create a GitHub Jenkins pipeline to demonstrate installing dependencies. *(25 Marks)*

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url:
 ↪  'https://github.com/<your-username>/<your-node-repo>.git'
```

```
            }
        }
        stage('Install Dependencies') {
            steps {
                sh 'node --version'
                sh 'npm --version'
                sh 'npm install'
            }
        }
    }
    post {
        success { echo 'Dependencies installed successfully' }
        failure { echo 'Dependency installation failed' }
    }
}
```

Expected console output: the Node/npm versions, then added `N` packages, then `Dependencies installed successfully`.

J3. Node project - start the application

Question: Consider the given Node.js project and create a GitHub Jenkins pipeline to demonstrate starting the application. *(25 Marks)*

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url:
 ↪  'https://github.com/<your-username>/<your-node-repo>.git'
            }
        }
        stage('Install Dependencies') {
            steps { sh 'npm install' }
        }
        stage('Start Application') {
            steps {
                withEnv(['BUILD_ID=dontKillMe',
                 ↪  'JENKINS_NODE_COOKIE=dontKillMe']) {
                    sh 'nohup npm start > app.log 2>&1 &'
                }
                sh 'sleep 3'
                sh 'cat app.log'
                sh 'curl -s http://localhost:3000 || echo "App not reachable"'
            }
        }
    }
}
```

Expected console output: `app.log` shows `Server running on port 3000` and `curl` prints the app's page.

J4. Python project - send reports by email

Question: Consider the given Python project available on your machine and create a GitHub Jenkins pipeline to send reports by email. *(25 Marks)*

Needs the **Email Extension** plugin and SMTP details configured in *Manage Jenkins -> System -> Extended E-mail Notification* (for example Gmail `smtp.gmail.com`, port 465, SSL, with an app password).

```
pipeline {
    agent any
    stages {
        stage('Checkout') {
            steps {
                git branch: 'main',
                    url:
 ↪  'https://github.com/<your-username>/<your-python-repo>.git'
            }
        }
        stage('Install') {
            steps {
                sh 'python3 -m venv venv'
                sh '. venv/bin/activate && pip install -r requirements.txt
 ↪  pytest'
            }
        }
        stage('Test & Report') {
            steps {
               sh '. venv/bin/activate && pytest --junitxml=report.xml || true'
            }
        }
    }
    post {
        always {
            junit 'report.xml'
            emailext(
                subject: "Build #${env.BUILD_NUMBER} -
 ↪  ${currentBuild.currentResult}",
                body: "Test report attached.\nBuild URL: ${env.BUILD_URL}",
                to: 'you@example.com',
                attachmentsPattern: 'report.xml'
            )
        }
    }
}
```

Expected result: the build page shows the test results, and the recipient gets an email with `report.xml` attached.

7. Viva Questions and Answers (Q3 - 10 Marks)

Shell Scripting

1. **What is a shell script?** A text file of shell commands run in order by the shell (e.g. Bash), used to automate tasks.
2. **What does `#!/bin/bash` mean?** The "shebang": it tells the system to run the script with Bash.
3. **How do you make a script executable?** `chmod +x script.sh`, then `./script.sh`.
4. **What are `$0`, `$1`, `$#`, `$?`?** Script name, first argument, number of arguments, exit status of the last command.
5. **Difference between `wc -l`, `wc -w`, `wc -c`?** Count lines, words, bytes (`-m` counts characters).
6. **What is a pipe?** `|` sends one command's output as the next command's input, e.g. `ls | wc -l`.
7. **What does `sed -i '/^$/d'` do?** Deletes empty lines from the file in place.
8. **Difference between `>` and `>>`?** `>` overwrites a file, `>>` appends.

Git & GitHub

1. **Git vs GitHub?** Git is the version control tool; GitHub is a cloud service that hosts Git repositories.
2. **`git clone` vs `git pull`?** Clone copies a whole repository the first time; pull fetches and merges new changes into an existing copy.
3. **What is `git fetch`?** Downloads remote changes without merging them.
4. **Lightweight vs annotated tag?** Lightweight is just a name for a commit; annotated stores author, date and a message.
5. **What is a webhook?** An HTTP callback: GitHub POSTs event data to your URL when something happens (e.g. a push).
6. **Why use ngrok?** It exposes a local port to the internet through a public URL so GitHub can reach `localhost`.
7. **What is staging (`git add`)?** Choosing which changes go into the next commit.
8. **How to see history?** `git log` (`--oneline` for short form).

GitHub Actions

1. **What is GitHub Actions?** GitHub's built-in CI/CD service that runs workflows on events.
2. **Where are workflows stored?** `.github/workflows/*.yml`.
3. **Workflow, job, step?** A workflow contains jobs; a job runs on a runner and contains steps; a step is a command or a reusable action.
4. **What does `on: push: branches: [main]` do?** Triggers the workflow on every push to `main`.
5. **What is `actions/checkout`?** Downloads the repository code onto the runner.
6. **What are artifacts?** Files saved from a run (reports, builds) that you can download afterwards.

Docker

1. **What is Docker?** A platform to package applications with their dependencies into portable containers.
2. **Image vs container?** An image is a read-only template; a container is a running instance of it.

3. **Container vs virtual machine?** Containers share the host kernel and are lightweight; VMs include a full guest OS.
4. **What is a Dockerfile?** A text file of instructions to build an image.
5. **`CMD` vs `RUN`?** `RUN` executes while building the image; `CMD` is the command run when the container starts.
6. **What does `-p 3000:3000` do?** Maps host port to container port.
7. **`docker rm` vs `docker rmi`?** Removes a container vs removes an image.
8. **`docker exec` vs `docker run`?** `exec` runs a command in an existing running container; `run` creates a new container.
9. **What is Docker Hub?** The public registry for sharing images.

Docker Compose

1. **What is Docker Compose?** A tool to define and run multi-container apps from one YAML file.
2. **Key commands?** `docker compose up -d`, `ps`, `logs`, `down`.
3. **Why use volumes for MySQL?** To keep database data after the container is removed.
4. **How is the MySQL user created?** Through `MYSQL_USER` and `MYSQL_PASSWORD` environment variables on first start.

Jenkins

1. **What is Jenkins?** An open-source automation server for CI/CD.
2. **What is a Jenkinsfile?** A text file defining the pipeline as code.
3. **Declarative vs scripted pipeline?** Declarative uses a structured `pipeline { }` syntax; scripted uses free-form Groovy in `node { }`.
4. **What is a stage / step?** A stage is a named phase (Build, Test); steps are the commands inside it.
5. **What is CI/CD?** Continuous Integration (merge and test often) and Continuous Delivery/Deployment (release automatically).
6. **How can Jenkins start on every push?** A GitHub webhook to `http://<jenkins>/github-webhook/` plus the *GitHub hook trigger* option.
7. **Why `BUILD_ID=dontKillMe`?** Without it Jenkins kills background processes started by the build when the build ends.